Track 5: Quantum Key Distribution

BB84, BBM92, eavesdroppers and secure key extraction

Qiskit Fall Fest Hackathon · Oct 9, 2026 · IISER Bhopal (EECS Club and Physics Club)

Read this first. You have until Oct 17, a laptop, and AI assistants. The ten problem statements are suggestions, not a menu you must order from. You may pick one, extend one, combine several, or invent something more ambitious. Original, advanced and cross-track projects are welcome and will be judged on the same criteria.

What we reward. A project that works, that you can explain line by line, and that is honestly compared against a classical alternative. Using AI to write code is allowed and encouraged; being unable to explain or verify that code is not.

How to read the problem statements

Every problem has the same parts: a short background, the problem itself, the core tasks (what a complete submission does), stretch goals (what makes it excellent), and a hint on tools. Treat the core tasks as the minimum, not the ceiling.

Simulators first, hardware as a bonus. Everything here can be done on a simulator (Qiskit Aer or the statevector tools). Running a small version on real IBM hardware, with a clear comparison against the simulator, earns extra credit in any track.

This track contains: **Problem 9**.

Problem 9: BB84 and BBM92 in Qiskit

Background

Quantum key distribution lets two people, Alice and Bob, agree on a secret key whose secrecy is guaranteed by physics: an eavesdropper who measures the qubits disturbs them, and the disturbance shows up as errors. In BB84, Alice sends single qubits prepared in randomly chosen bases. In BBM92, a source distributes entangled pairs and Alice and Bob each measure their half in randomly chosen bases; the entanglement does the job of the preparation.

Problem

Implement both protocols end to end in Qiskit, attack them with an eavesdropper, and show how secure key is extracted from noisy, partly compromised data.

Core tasks (the minimum for a complete submission)

1. Implement BB84: random bits and bases for Alice, random measurement bases for Bob, and the sifting step that keeps only matching bases. Report the sifted key rate (about half of the qubits in the ideal case).
2. Implement BBM92 with Bell pairs and the same sifting logic. Show that it gives the same kind of key by a different route.
3. Add an intercept-and-resend eavesdropper (Eve) and show that she raises the error rate in the sifted key (the quantum bit error rate, or QBER) to around 25 percent when she attacks every qubit. Plot QBER against the fraction of qubits Eve attacks.
4. Add a channel noise model and a decision rule: when is the QBER low enough to keep the key, and when must the run be aborted?

Stretch goals (what makes it excellent)

- Implement error estimation on a sampled subset, error correction (reconciliation) and privacy amplification, so the output is a shorter final key. Estimate the final key rate against QBER.
- For BBM92, run a Bell-inequality (CHSH) test as a second security check and show that Eve's attack lowers the violation.
- Model losses and imperfect detectors, and study the distance dependence of the key rate.
- Replace reconciliation by the LDPC decoding built in Problem 8, or protect the transmission with a Steane-coded logical qubit from Problem 7.

Hints on tools

Simulating qubit by qubit with Aer is fine for a few thousand qubits; for larger runs, you can simulate the statistics directly. Be clear in your write-up about what is a physical simulation and what is a shortcut. A security argument that you can state in two sentences is part of the deliverable.

Go beyond: combine tracks or invent your own

The most impressive entries will not stop at one box. None of these is required, and you are free to propose something entirely different.

Ideas that involve Track 5

Idea	Tracks combined	What makes it interesting
A logical qubit inside a QKD protocol	Steane code and BB84 or BBM92	Does encoding the transmitted qubits lower the QBER, and at what cost in qubits?
LDPC reconciliation in QKD post-processing	LDPC decoding and QKD	Replace a simple error-correction step with your own decoder, and plot final key rate against QBER.
Quantum-safe key exchange pipeline	Lattice cryptography and QKD	Compare a lattice-based key exchange with QKD in security assumptions, cost and failure modes.

Open invitation. If you have your own idea, such as a quantum game, an algorithm tutor, a benchmark of a quantum algorithm from our workshop (Grover, phase estimation, the QFT), or a tool that makes any of the above easier for the next student, propose it. Write three sentences on the problem, the method, and how you will know it worked, and check with the organisers early.

How to scope a combined project. Get the smallest version of each part working separately first, then connect them. A small, honest, working combination beats an ambitious one that does not run.

What every team submits

Whichever problem you choose (or invent), your submission has the same four parts:

- A code repository or notebook that runs from a clean start, with versions of the packages listed.
- A write-up PDF (two to four pages) covering the problem, method, results and an honest limits section.
- At least one figure that shows the main result.
- A short AI-use note: which tools you used, one thing the AI got wrong, and how you caught it.

Bonus. Bonus points are available for a hardware run on a real IBM backend, with a clear comparison against the simulator.

Presentation. Submissions close on Oct 17, 2026, 12:00 noon. After reviewing them, the organisers invite selected teams to present on Oct 18, 2026: a five-minute demo followed by questions, where every team member must be able to take questions. Teams that are not invited are still fully judged on their submission.

Pitch deck (advised). Teams are selected from their submissions alone; a pitch deck is not part of the submission and is not used for selection. However, selected teams will have less than 24 hours between the invitation and the presentation, so we strongly advise preparing a short pitch deck for your idea in advance.

Full submission rules are in *docs/guidelines_and_submission_rules.pdf*.

How you are judged

The same scoring applies to every problem and to original projects alike.

Criterion	Points	What judges look for
Correctness and verification	30	Results checked against an exact or classical reference, tests, sensible error bars
Understanding and insight	25	Clear explanation of why the method works or fails, shown in the write-up (and in the live questions for invited teams)
Honest comparison and limits	20	A fair classical baseline, stated assumptions, no unsupported claims of quantum advantage
Ambition and originality	15	Stretch goals, combined tracks, or a new idea beyond the suggested statements
Clarity of code and write-up	10	Readable, reproducible code and a clear write-up and story
Total	**100**	Bonus points are available on top for a real IBM hardware run (see Bonus).

> **On using AI.** AI assistants are welcome at every step. The AI-use note and the presentation questions exist because the person submitting is responsible for the result: you should be able to explain each circuit, formula and plot, and show how you verified it.